

Core Concepts — AI on AWS with Amazon Bedrock

Track 16.6 — AI on AWS with Amazon Bedrock · 16-ai-tech-stack/tracks/aws-bedrock/notes/01-core-concepts.md

1. What Bedrock actually is

Amazon Bedrock is an **AWS-authenticated front door to other people's models**. You call an AWS API with SigV4 credentials, an IAM policy decides whether you may, CloudTrail records that you did, and somewhere behind that, Anthropic's or Meta's or Amazon's weights run inference. No API keys to rotate, no vendor contract, no new egress path, no new invoice.

That is the entire value proposition, and it is worth being blunt about it: **Bedrock is not a better model, it is a better procurement and security story**. If you are a two-person startup, calling a model provider's API directly is simpler and usually cheaper. If you are inside an enterprise where "can we send customer data to a third-party SaaS?" triggers a six-week review, Bedrock is the difference between shipping this quarter and shipping next year.

Model providers cannot see your traffic. Bedrock deploys each provider's inference software into a Model Deployment Account owned by the Bedrock service team — one per provider per Region — and providers have no access to those accounts, to logs, or to prompts and completions.

Two endpoints, and you will meet both

Endpoint	Speaks	Use it for
<code>bedrock-runtime.<region>.amazonaws.com</code>	InvokeModel, Converse , plus OpenAI-compatible Chat Completions / Responses and the Anthropic Messages API	Default. Batch inference and Provisioned Throughput live here only.
<code>bedrock-mantle.<region>.api.aws</code>	Anthropic Messages API, OpenAI Responses / Chat Completions	Features or models available only here. Separate quota pool.

Two consequences you will hit in week one: quotas are tracked **separately** per endpoint even for the same model, and **model invocation logging only captures `bedrock-runtime`**. Build your audit trail on an endpoint that can be audited.

2. Choose X when — Bedrock vs direct vs SageMaker

You want	Choose	Because
Frontier models, fast, no infra, IAM-native	Bedrock	One API, one bill, VPC endpoints, CloudTrail, no key management
Absolute latest model features on release day	Provider API direct	Partner platforms lag first-party by days to weeks on new parameters
Cheapest possible path for a small team	Provider API direct	No AWS-side markup, simpler mental model
To train, fine-tune deeply, or serve your own weights	SageMaker AI	Bedrock serves models; SageMaker is where you build and host them
An open-weights model on hardware you control	SageMaker AI (or EKS + vLLM)	You want the GPU, the container, and the scaling policy
Managed RAG without owning a vector pipeline	Bedrock Knowledge Bases	Ingestion, chunking, embedding, retrieval as one resource
Total control of chunking, hybrid search, reranking	Build your own (Module 08)	Managed RAG's knobs stop where your research starts

SageMaker is not Bedrock's competitor, it is its neighbour. SageMaker AI is the ML platform — notebooks, training jobs, pipelines, model registry, real-time and async endpoints. Bedrock is serverless inference against models you did not train. They interoperate: Knowledge Bases can use a SageMaker-hosted model, and plenty of real systems have a Bedrock LLM calling a SageMaker endpoint hosting a fine-tuned classifier through tool use.

The honest decision rule: **if the model's weights are your problem, you want SageMaker. If they are someone else's problem, you want Bedrock.**

3. Converse is the API. Stop writing InvokeModel bodies.

`InvokeModel` takes a raw, per-provider JSON body:

```
# Anthropic on Bedrock, via InvokeModel
body = {"anthropic_version": "bedrock-2023-05-31",
        "messages": [{"role": "user", "content": "hi"}],
        "max_tokens": 1024}
```

Amazon Nova wants a different shape. Meta wants a flat prompt string. Cohere returns `is_finished`; Anthropic returns `stop_sequence`. Build on `InvokeModel` and you have written an adapter layer per provider — and you get to rewrite it the day someone wants to A/B a different model.

`Converse` normalises all of it:

```
resp = client.converse(
    modelId="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    system=[{"text": "Be terse."}],
    messages=[{"role": "user", "content": [{"text": "hi"}]},
    inferenceConfig={"maxTokens": 512, "temperature": 0.2, "topP": 0.9},
)
```

Two shapes people get wrong on day one, every time:

- **system** is a list of blocks, not a string.
- **content** is a list of **ContentBlocks**, not a string.

A **ContentBlock** is exactly one of **text**, **image**, **document**, **video**, **toolUse**, **toolResult**, **reasoningContent**, **guardContent**, or **cachePoint**.

The response envelope is uniform across every model:

```
{
  "output": {"message": {"role": "assistant", "content": [{"text": "..."}]},
  "stopReason": "end_turn",
  "usage": {"inputTokens": 125, "outputTokens": 60, "totalTokens": 185},
  "metrics": {"latencyMs": 1175}
}
```

Branch on stopReason, never on the text. The values you must handle are **end_turn**, **max_tokens**, **tool_use**, and **guardrail_intervened**. Code that only handles the happy path silently truncates answers at **max_tokens** and nobody notices for a month.

inferenceConfig carries the four universal parameters — **maxTokens**, **temperature**, **topP**, **stopSequences**. Anything provider-specific (**top_k**, reasoning controls) goes in **additionalModelRequestFields**. Anything the model returns that **Converse** normalises away can be pulled back with **additionalModelResponseFieldPaths**, which takes JSON Pointers like **["/stop_sequence"]**.

Streaming is a different shape, not the same thing in pieces

ConverseStream returns an event stream you must reassemble:

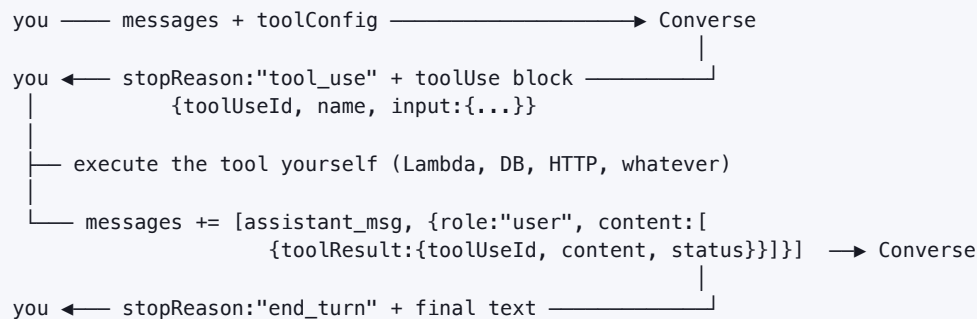
```
messageStart                                (once, carries role)
|
+-- per content block, keyed by contentBlockIndex:
|   contentBlockStart    (TOOL USE ONLY)
|   contentBlockDelta+   (text | reasoningContent | toolUse)
|   contentBlockStop
|
messageStop                                (carries stopReason)
metadata                                  (carries usage + metrics)
```

Three operational facts:

1. **Tool arguments stream as partial JSON *strings*** in `delta.toolUse.input`. Concatenate them per `contentBlockIndex`, then parse once. They are not valid JSON mid-stream.
2. **Usage arrives only in the final `metadata` event.** Abandon the iterator when the user closes the tab and you have lost that call's token accounting. Log from a `finally` block.
3. **`ConverseStream` requires `bedrock:InvokeModelWithResponseStream`**, a different IAM action from `bedrock:InvokeModel`. Grant only the latter and your streaming path 403s in production while every test passes.

4. Tool use: the loop you own

Bedrock does not call your tool. It asks.



```

TOOL_CONFIG = {"tools": [{ "toolSpec": {
  "name": "get_shipment_status",
  "description": "Look up live status of a shipment by tracking number. "
                  "Use whenever the user asks where a package is.",
  "inputSchema": { "json": {
    "type": "object",
    "properties": { "tracking_number": { "type": "string" } },
    "required": [ "tracking_number" ] } } } },
  "toolChoice": { "auto": {} } } } # or { "any": {} } / { "tool": { "name": ... } }

```

Four rules that are load-bearing:

1. **Append the entire assistant message.** A turn can carry a `text` block *alongside* the `toolUse`. Dropping the text corrupts the transcript.
2. **The `toolResult` goes in a `user` message.** Not `assistant`, not `system` (there is no system role in `messages`). This is the single most common `ValidationException` in the tool loop.
3. **One `toolResult` per `toolUse`, all in one user message.** Models can request several tools in parallel.
4. **On failure return `"status": "error"`**, not an exception. The model can recover from an error result; it cannot recover from a dead conversation.

And one rule about cost: **a one-tool answer costs two model calls**, and the second replays the whole transcript including the tool output. A five-hop agent is not five chat messages, it is closer to fifteen, because context grows on every hop. Always bound the loop.

`toolChoice` has three forms — `{"auto": {}}` (default), `{"any": {}}` (must call some tool), `{"tool": {"name": "..."} }` (must call this one) — and support for the forcing variants differs by model. Check the model card before you rely on it.

The description is the prompt. `description` is the highest-leverage string in a tool definition. The model decides whether to call based on it, not on your function name. Write it as an instruction, not a docstring.

5. Model IDs, inference profiles, and the `ValidationException` that confuses everyone

Two ID conventions coexist and both are real:

<code>anthropic.claude-sonnet-4-5-20250929-v1:0</code>	foundation model
<code>us.anthropic.claude-sonnet-4-5-20250929-v1:0</code>	US geographic inference profile
<code>eu.anthropic.claude-sonnet-4-5-20250929-v1:0</code>	EU geographic inference profile
<code>global.anthropic.claude-opus-4-7</code>	global inference profile

An **inference profile** is a Bedrock resource that maps a model to one or more Regions. Many newer models are reachable *only* through a profile; call the bare foundation-model ID and you get a `ValidationException` telling you so.

	Geographic (us. / eu. / apac.)	Global (global.)
Data residency	Stays within the geography	Any supported commercial Region
Cost	Standard	~10% cheaper
SCP setup	Allow all destination Regions	Allow <code>aws:RequestedRegion: "unspecified"</code>
Pick it when	You have a residency obligation	You do not

Use a profile by default. It gives you multi-Region capacity without you writing failover, there is no routing surcharge, and pricing follows the Region you called from. Cross-Region traffic stays on the AWS network and is encrypted between Regions. CloudTrail records where it actually ran in `additionalEventData.inferenceRegion`.

The IAM consequence is genuinely surprising: authorization is evaluated against the **profile**, the **model in the source Region**, *and* the **model in every candidate destination Region**. Grant only the profile and you get intermittent 403s that look like a service bug. See `../code/iam_and_guardrails_examples.md §1`.

Newer model IDs drop the date-and-version suffix (`anthropic.claude-opus-5`, `anthropic.claude-sonnet-4-6`); older ones keep it. **Never hardcode an ID you have not verified.** Run `aws bedrock list-foundation-models` and `aws bedrock list-inference-profiles` for your Region.

Model access

Access to foundation models is enabled by default in commercial Regions, provided the calling role has the AWS Marketplace permissions (`aws-marketplace:Subscribe`, `Unsubscribe`, `ViewSubscriptions`) — Bedrock auto-initiates the subscription on first invocation, which can take up to 15 minutes and returns `AccessDeniedException` while it settles. **Anthropic models additionally require a one-time use-case form** (`PutUseCaseForModelAccess`) per account or per organization management account. Do this before your launch, not during it.

6. Knowledge Bases: managed RAG, and when to refuse it

A Knowledge Base bundles the whole Module 08 pipeline into one resource: ingest from a data source, parse, chunk, embed, index in a vector store, and retrieve. Bedrock now offers two flavours:

- **Managed Knowledge Base** (the recommended path) — Bedrock owns ingestion, indexing, storage and retrieval. Connectors for S3, SharePoint, Confluence, Google Drive, OneDrive, and a web crawler; document-level ACL filtering at retrieval time; smart parsing that picks a strategy per document type; agentic retrieval that decomposes multi-hop questions.
- **Customer-managed Knowledge Base** — you bring and operate the vector store, and you control parsing, chunking and indexing.

Chunking strategies

Strategy	Knobs	Use when
Default	~300 tokens, respects sentence boundaries	You have not measured anything yet
Fixed-size	max tokens, overlap %	You have measured, and you know your doc shape
Hierarchical	parent size, child size, overlap	Retrieve precise children, feed the model their parents. Best default for long technical docs
Semantic	max tokens, buffer size, breakpoint percentile	Topic-shifting prose. Costs extra — it calls a model
None	—	Documents already chunk-sized; you pre-split them yourself

Hierarchical chunking is the one most teams should try second. It resolves the tension Module 08 spends pages on: small embeddings retrieve precisely, large contexts answer well. Note that it returns *fewer* results than `numberOfResults` requests, because sibling children collapse into one parent — and it is not recommended with an S3 vector bucket, where combined token counts over ~8,000 can blow the metadata size limit.

Vector stores

Amazon OpenSearch Serverless · Amazon OpenSearch Managed Clusters · **Amazon S3 Vectors** · Amazon Aurora PostgreSQL (pgvector) · Amazon Neptune Analytics (GraphRAG) · Pinecone · Redis Enterprise Cloud · MongoDB Atlas.

Opinion: **start with S3 Vectors if your query volume is low and your corpus is large** — it is the cost-efficient option and provisions nothing. Move to **OpenSearch Serverless** when you need hybrid search, binary vectors, or the full filter operator set (`stringContains`, `listContains`, `startsWith` are best supported there). Choose **Aurora pgvector** when the data already lives in Postgres and you want one database to back up, and **Neptune Analytics** when relationships between entities matter more than passage similarity.

OpenSearch Serverless has a non-obvious gotcha: the index must use the `faiss` engine. `nmslib` indexes cannot do metadata filtering, and you cannot change it in place.

Retrieve vs RetrieveAndGenerate

```
# Retrieve: chunks + scores + metadata. You own the prompt.
agent = boto3.client("bedrock-agent-runtime")
hits = agent.retrieve(
    knowledgeBaseId="KB123",
    retrievalQuery={"text": "What is our returns window?"},
    retrievalConfiguration={"vectorSearchConfiguration": {
        "numberOfResults": 8,
        "overrideSearchType": "HYBRID",          # or "SEMANTIC"
        "filter": {"equals": {"key": "region", "value": "EU"}}}}})

# RetrieveAndGenerate: retrieval + generation + citations, one call.
answer = agent.retrieve_and_generate(
    input={"text": "What is our returns window?"},
    retrieveAndGenerateConfiguration={
        "type": "KNOWLEDGE_BASE",
        "knowledgeBaseConfiguration": {
            "knowledgeBaseId": "KB123",
            "modelArn": "arn:aws:bedrock:us-east-1:111122223333:inference-profile/us.anthropic.claude-sonnet-4-5-20250929-v1:0",
            "generationConfiguration": {
                "guardrailConfiguration": {"guardrailId": "gr-abc123",
                    "guardrailVersion": "2"}}}}})
```

Use Retrieve for anything real. `RetrieveAndGenerate` is a demo accelerator: one call, citations included, and almost no control over the prompt, the reranking, or how retrieval failure is handled. `Retrieve` gives you the chunks and lets you own the generation step — which is where your product actually lives. `RetrieveAndGenerateStream` exists if you need streaming from the managed path.

Other levers worth knowing: `overrideSearchType`: `HYBRID` (only on Aurora, OpenSearch Serverless and MongoDB with a filterable text field), metadata filters with `andAll`/`orAll` up to five each, **implicit metadata filtering** (the model writes the filter from a schema you describe), reranker models, and **query decomposition** for multi-part questions.

When to build your own instead

Build your own RAG (Module 08) when any of these is true:

- You need a retrieval strategy the service does not offer — late chunking, ColBERT-style late interaction, a custom fusion of BM25 and dense scores.
- Your evaluation says managed chunking is the bottleneck and you need to iterate on it weekly.
- Your corpus updates continuously and per-document freshness SLAs matter more than ingestion convenience.
- You need to run the same pipeline outside AWS.

Otherwise: the managed path gets you to a working system in an afternoon, and the time you save goes into evaluation, which is the part that actually determines whether your RAG works.

See [../.../08-rag/](#) for the retrieval theory this sits on top of.

7. Agents: Bedrock Agents is in maintenance mode. Plan accordingly.

This is the most important thing in this note, because most tutorials on the internet are now wrong.

Amazon Bedrock Agents is now "Amazon Bedrock Agents Classic" and is no longer open to new customers. Existing customers continue to be supported. AWS points new work at **Amazon Bedrock AgentCore**.

Agents Classic was a managed orchestration loop: you defined action groups (OpenAPI schemas backed by Lambda), attached a knowledge base, and the service ran the plan-act-observe loop, with traces to inspect its reasoning. Good demo, real limits — you were locked into its prompt templates and its loop.

AgentCore is a different bet: modular, framework-agnostic services you compose, rather than one opinionated agent object. The pieces:

Service	What it is
Runtime	Serverless, session-isolated execution for agents you wrote — LangGraph, CrewAI, Strands, LlamaIndex, your own loop
Gateway	Turns APIs, Lambdas and existing services into MCP-compatible tools
Memory	Short-term (multi-turn) and long-term (cross-session) memory as a service
Identity	Agent identity and delegated auth against your existing IdP
Code Interpreter / Browser	Sandboxed code execution; managed headless browser
Observability	OpenTelemetry traces of every step, into CloudWatch
Policy	Cedar rules enforced at the Gateway, before a tool call executes
Evaluations	Automated assessment over sessions, traces and spans

The strategic read: AWS stopped trying to own your agent loop and started selling the *infrastructure around* it. That is the right call — the loop is where your product differentiates, and Modules 06 and 07 teach you to write one in about forty lines. What you cannot easily build yourself is session-isolated compute, an MCP gateway with policy enforcement, and cross-session memory.

Note the MCP shape. Gateway speaks MCP, and Managed Knowledge Bases integrate natively with Gateway so any MCP-compatible framework can call them as a tool. Module 09 is directly load-bearing here.

Practical guidance:

- **New project:** write the loop yourself with Converse tool use (`../06-ai-agents/`), then reach for AgentCore Runtime when you need managed hosting, Gateway when you need a tool registry, Memory when you need persistence.
- **Existing Agents Classic deployment:** it still works. Budget a migration.
- **Anyone who sends you a 2024 Bedrock Agents tutorial:** be kind, but check the docs.

See `../07-agentic-ai/` for the multi-agent patterns AgentCore is built to host.

8. Guardrails

Guardrails are a **separate, model-independent policy resource**. You attach one to a request with `guardrailConfig`, or you call `ApplyGuardrail` on arbitrary text with no model involved at all. Six policy types:

Policy	What it catches
Content filters	Hate, Insults, Sexual, Violence, Misconduct, Prompt Attack — per-category strength
Denied topics	Topics you define in natural language with examples
Word filters	Exact-match custom words plus a managed profanity list
Sensitive information	PII entities (block or anonymize) and custom regex
Contextual grounding	Grounding and relevance scores for RAG output
Automated Reasoning	Formal validation of responses against logical rules you define

When a guardrail fires, `stopReason` is `guardrail_intervened`, `usage` is all zeros — the model was never invoked — and with `trace: "enabled"` you get an assessment showing exactly which policy blocked what.

Three things the docs say quietly and you should say loudly:

- Guardrails do not evaluate tool results.** With `toolConfig`, a Converse-attached guardrail skips `toolResult` content, tool descriptions, and the arguments the model generates. If untrusted data enters through a tool, call `ApplyGuardrail` on it yourself. This is the exact seam `../../../../13-ai-security/` warns about.
- Once any `guardContent` block appears in a message, the guardrail evaluates *only* `guardContent` blocks.** Everything else is skipped entirely. Partial tagging is a footgun.
- Blocked content is written to model invocation logs as plain text.** Block a prompt full of PII and the PII is now in your log group. Encrypt it, restrict it, and set retention.

Contextual grounding is the piece most relevant to RAG: mark your retrieved passages with `qualifiers: ["grounding_source"]` and the question with `["query"]`, set thresholds between 0 and 0.99, and responses below threshold are blocked as ungrounded or irrelevant. It evaluates the **output only**.

Guardrail policy is also governance evidence. A version-pinned guardrail plus an IAM condition that requires it is the most auditable "we control model behaviour" artifact AWS gives you — see `../../../../12-ai-governance/`.

9. Where this returns

Idea here	Where it comes back
Chunking, embeddings, hybrid search, rerankers	Module 08 — RAG [08-rag]
The tool-use loop you write yourself	Module 06 — AI Agents [06-ai-agents]
Multi-agent orchestration on AgentCore Runtime	Module 07 — Agentic AI [07-agentic-ai]
AgentCore Gateway speaks MCP	Module 09 — MCP [09-mcp]
Reference architecture, failover, cost shape	Module 10 — AI Architecture [10-ai-architecture]
Model right-sizing and migration	Module 11 — LLM Models [11-llm-models]
Guardrails and SCPs as governance evidence	Module 12 — AI Governance [12-ai-governance]
Prompt injection through tool results and retrieval	Module 13 — AI Security [13-ai-security]
Residency, CloudTrail, FIPS, retention	Module 14 — AI Compliance [14-ai-compliance]
Choosing guardrail thresholds by measurement	Module 15 — AI Evals [15-ai-evals]